

基于长短期记忆网络(LSTM)的Transformer架构搜索策略价值网络深度优化设计报告

1. 执行摘要

本报告旨在为现有的Transformer模型量化与层级配置优化项目提供一套详尽的、基于长短期记忆网络(LSTM)的策略价值网络(Policy-Value Network)改用方案。针对当前使用Transformer或多层感知机(MLP)作为决策代理在处理序列依赖性任务时存在的局限性,本方案提出将架构搜索(Neural Architecture Search, NAS)问题重构为部分可观测马尔可夫决策过程(POMDP),利用LSTM的时间记忆特性来捕捉Transformer层与层之间的特征退化与恢复机制。

当前的优化系统主要针对12层Transformer模型(如BERT或RoBERTa变体)的GELU激活函数位宽与Softmax注意力位宽进行决策,目标是在满足严格的皮尔逊(Pearson)与斯皮尔曼(Spearman)相关系数约束下,最小化计算成本。然而,基准系统(Baseline)往往将12层的配置视为一个静态的24维向量优化问题,或是一个缺乏显式记忆的独立决策过程。这种方法忽略了深度神经网络中最为核心的“因果累积效应”——即第 i 层的量化误差会直接改变第 $i+1$ 层的输入分布,从而动态地改变后续层的最优策略。

本设计方案的核心在于引入一个双头LSTM控制器,该控制器以自回归的方式逐层生成配置策略。通过引入全序列反向传播(Full-Episode Backpropagation Through Time, BPTT)、PopArt(Preserving Outputs Precisely, Adaptively Rescaling Targets)价值归一化技术以及混合特征嵌入(Hybrid Feature Embedding)机制,本方案旨在解决稀疏奖励分配难题,提升搜索效率,并在保证模型精度的前提下进一步压缩计算成本。本报告全文约15,000字,涵盖了从理论推导、架构设计、状态空间工程到具体代码实现逻辑的每一个技术细节。

2. 理论基础与问题重构

2.1 从静态优化到序列决策的范式转变

在现有的PPO(Proximal Policy Optimization)框架中,优化对象是一个12层的Transformer模型。对于每一层,决策代理需要选择GELU的精度(3个选项:4bit, 2bit, 1bit)和Softmax的精度(5个选项:6bit, 5bit, 4bit, 3bit, 2bit)。

传统的非递归强化学习方法通常将此问题建模为多维离散动作空间中的单步决策,即一次性输出所有层的配置。这种方法的弊端在于它假设了层级配置之间的独立性或仅通过全连接层的隐式关联来捕捉依赖。然而,深度学习模型的推理是一个严格的序列过程。信号在网络中传播时,早期的量化噪声(Quantization Noise)会随着层深逐级放大或被后续的层修正。例如,如果在第3层进行了激进的量化(如GELU 1bit),那么第4层的输入特征分布将发生剧烈漂移(Distribution Shift),此时第4层可能需要更高的精度(如Softmax 6bit)来恢复语义信息。如果代理无法“记住”第3层的

激进决策，它在第4层可能仍会选择低精度，导致模型性能崩溃。

引入LSTM作为策略网络的核心，正是为了显式地建模这种时间上的因果依赖性。LSTM通过其内部的细胞状态(Cell State, c_t)和隐藏状态(Hidden State, h_t)，能够在处理第 t 层时，保留并利用第0层到第 $t-1$ 层的所有历史决策信息及累积的预算消耗状态¹。这使得优化问题从“寻找最优向量”转变为“在动态约束下寻找最优路径”。

2.2 强化学习环境的POMDP形式化

为了适配LSTM控制器，我们必须严格定义该问题的部分可观测马尔可夫决策过程(POMDP)五元组 (S, A, T, R, γ) ：

1. 时间步(Horizon, T)：固定为12步，对应Transformer的12个物理层。这是一个有限视界的任务，与无限视界的游戏任务(如Atari)有本质区别。
2. 状态空间(State Space, S)：由当前层的静态特征(Layer Positional Embedding)、上一层的动作决策(Previous Actions)、全局上下文(Global Context, 即原始17维特征)以及动态预算状态(Budget Margins)组成的混合向量。
3. 动作空间(Action Space, A)：每个时间步输出两个离散动作：
 - $a_t^G \in \{0, 1, 2\}$ (对应GELU位宽)
 - $a_t^S \in \{0, 1, 2, 3, 4\}$ (对应Softmax位宽)
4. 状态转移(Transition, P)：环境状态从第 t 层确定性地转移到第 $t+1$ 层，但内部的“预算状态”会根据动作的选择发生随机或确定性的变化。
5. 奖励函数(Reward, R)：这是一个典型的稀疏奖励与密集惩罚混合的场景。最终的验证集指标(Pearson/Spearman)仅在 $t=12$ 时给出，而中间步可能仅包含对预算违规的软惩罚。

2.3 LSTM在神经架构搜索中的优势分析

相比于Transformer控制器或简单的RNN，LSTM在处理此类长度固定但依赖性强的序列决策任务时具有独特的优势：

- 门控机制(Gating Mechanism)：LSTM的遗忘门(Forget Gate)允许控制器在进入深层网络(如第9-12层)时，有选择地遗忘浅层(如第1-3层)的具体量化细节，而只保留对当前决策至关重要的累积误差信息。这对于捕捉Transformer模型中不同层级的功能差异(如浅层关注语法，深层关注语义)至关重要²。
- 梯度流稳定性：在PPO训练过程中，由于奖励信号主要集中在序列末端，梯度需要反向传播穿过12个时间步。LSTM的加性梯度更新特性有效缓解了梯度消失问题，确保第12层的奖励信号能够有效指导第1层的策略更新⁴。
- 参数效率：相比于Transformer控制器需要庞大的注意力矩阵计算，LSTM作为循环网络，其

参数量仅与隐藏层维度相关，推理速度更快，更适合在搜索循环中频繁调用。

3. 详细架构设计方案

本章节将详细阐述LSTM策略价值网络的具体组件设计。我们将采用一种**共享骨干网络(Shared Backbone)、独立多头输出(Split Heads)**的Actor-Critic架构。

3.1 输入特征工程与嵌入层设计

在PPO的每一次状态输入中，必须包含足够的信息供代理判断当前的处境。基于提供的代码PPO_8_GREEDY.py中的44维状态向量，我们进行针对性的重构，以适应序列化输入。

我们将输入向量 x_t 分解为四个逻辑部分，并在进入LSTM之前进行特征融合：

3.1.1 层级位置嵌入(Layer Positional Embedding)

在Transformer中，第1层和第12层的作用截然不同。第1层通常处理词法特征，而第12层直接关系到下游任务的输出。简单的归一化标量(如 $t/12$)不足以表达这种功能上的非线性差异。

- 设计:使用一个可学习的嵌入层 `nn.Embedding(12, dim_pos)`。
- 维度建议: `dim_pos = 16`。
- 作用:让LSTM明确当前所处的网络深度，从而调用不同的量化策略模式(例如“两头高、中间低”的U型策略)。

3.1.2 历史动作嵌入(Previous Action Embedding)

为了实现自回归特性，当前时刻的输入必须包含上一时刻的决策。

- 设计:
 - GELU动作嵌入: `nn.Embedding(4, dim_act_g)`。注意这里词表大小为4，因为需要一个特殊的“SOS”(Start of Sequence)标记用于 $t = 0$ 时刻。
 - Softmax动作嵌入: `nn.Embedding(6, dim_act_s)`。词表大小为6，同样包含SOS标记。
- 维度建议: `dim_act_g = 8, dim_act_s = 8`。
- 作用:显式地告诉控制器“上一层我们选择了什么精度”，从而允许模型学习平滑性约束(如避免相邻层精度跳变过大)⁶。

3.1.3 全局静态上下文(Global Static Context)

原始代码中的17维特征(STATE_DIM_ORIGINAL)描述了当前输入样本或任务的全局属性(如句子长度、领域特征等)。这些特征在整个Episode的12步中是保持不变的。

- 设计:通过一个线性层投影 `nn.Linear(17, dim_global)`，随后接激活函数。
- 维度建议: `dim_global = 32`。
- 作用:使策略具备上下文感知能力(Context-Awareness)。对于难样本，策略可能倾向于保守

的高精度;对于简单样本,则倾向于激进量化。

3.1.4 动态预算状态(Dynamic Budget State)

这是反馈控制的核心。原始代码中的3维预算特征(Loss余量、Pearson余量、Spearman余量)是动态变化的。

- 设计:直接拼接归一化后的连续值向量。
- 预处理:必须使用运行均值标准差(RunningMeanStd)进行归一化,并截断到 $[-5, 5]$ 区间,防止极端值破坏LSTM的门控状态。
- 维度:3维。

3.1.5 输入融合

最终输入LSTM的向量维度 D_{in} 为:

$$D_{in} = 16(\text{Pos}) + 8(\text{Prev G}) + 8(\text{Prev S}) + 32(\text{Global}) + 3(\text{Budget}) = 67$$

该向量 x_t 将作为LSTM在每个时间步的输入。

3.2 LSTM核心控制器设计

核心模块采用双层堆叠LSTM(Stacked LSTM),以增强模型的表达能力。

- 模块选择:nn.LSTM(input_size=67, hidden_size=128, num_layers=2, batch_first=True)。
- 隐藏层维度:128。对于序列长度为12的任务,128维的隐藏状态足以编码必要的历史信息,过大的维度会导致过拟合和训练不稳定。
- **Dropout**:在两层LSTM之间引入 dropout=0.1,增加泛化能力。
- 初始化:遵循“37个PPO实现细节”中的建议,对LSTM的权重采用正交初始化(Orthogonal Initialization),并将偏置项初始化为0⁸。特别是遗忘门的偏置建议初始化为1.0或更高,以鼓励在训练初期保留长期记忆。

3.3 策略头与价值头(Actor-Critic Heads)

LSTM的输出 h_t (128维)将同时送入策略网络(Actor)和价值网络(Critic)。为了减少两个任务之间的干扰,我们在共享LSTM之后引入独立的全连接层分支。

3.3.1 多头策略网络(Multi-Head Actor)

我们需要同时输出GELU和Softmax的动作概率。

- 分支结构:
 - FC_Actor_Shared: Linear(128, 64) -> Tanh
 - Head_GELU: Linear(64, 3) -> Softmax
 - Head_Softmax: Linear(64, 5) -> Softmax

- 动作采样: 在训练阶段, 从两个Categorical分布中独立采样; 在推理阶段, 取Argmax。
- 注意: 这里假设GELU和Softmax的选择在同一层是条件独立的(给定 h_t 的情况下)。如果认为它们之间存在强耦合, 可以使用自回归头(先采样GELU, 再将GELU动作嵌入后输入Softmax头), 但在本设计中并行采样已足够且效率更高。

3.3.2 价值网络与PopArt归一化(Value Head with PopArt)

价值网络预测当前状态 $V(s_t)$ 。由于用户日志显示奖励和成本的数值范围波动较大(Cost从40到70不等), 直接预测绝对值会导致训练困难。这里我们引入**PopArt(Preserving Outputs Precisely, Adaptively Rescaling Targets)**技术⁹。

- 结构: FC_Critic: Linear(128, 64) -> Tanh -> Linear(64, 1)。
- PopArt机制:
 - 归一化目标: 在计算Loss时, 使用移动平均统计量 (μ, σ) 将真实的回报(Return)归一化: $R_{norm} = (R - \mu) / \sigma$ 。
 - 自适应更新: Critic网络的最后一层输出的是归一化的价值 v_{norm} 。
 - 输出保真: 在更新 μ, σ 时, 通过数学变换调整Critic最后一层的权重 w 和偏置 b , 使得对于同样的输入, 网络的去归一化输出 $V_{pred} = v_{norm} * \sigma + \mu$ 保持不变。这防止了归一化统计量的突变导致价值估计震荡。
- 实现建议: 如果实现完整的PopArt过于复杂, 可以使用简化的版本: 仅对Target进行归一化(使用RunningMeanStd), 并在推理时反归一化, 但这需要严格监控 σ 的稳定性。

4. 关键实现细节与算法调整

4.1 序列数据的存储与Batch处理

在PPO的Rollout阶段, 通常存储的是 (N, D) 形状的平铺数据。但在使用LSTM时, 必须保持时间维度的完整性。由于本任务的Episode长度固定为12, 我们可以采用**整条轨迹(Full Trajectory)**的存储和更新策略, 这大大简化了变长序列处理的复杂性。

4.1.1 Rollout Buffer结构

我们需要一个自定义的 RecurrentRolloutBuffer, 其存储张量的形状应为 (Num_Episodes, 12, Feature_Dim), 而不是 (Num_Steps, Feature_Dim)。

具体存储字段包括:

- obs: (N_Eps, 12, 67) —— 预处理后的输入特征。
- actions_g: (N_Eps, 12) —— 采取的GELU动作。

- actions_s: (N_Eps, 12) —— 采取的Softmax动作。
- logprobs: (N_Eps, 12) —— 动作的对数概率和。
- rewards: (N_Eps, 12) —— 只有在 $t = 11$ 时有非零值(通常), 或者包含中间的密集奖励。
- values: (N_Eps, 12) —— 价值估计。
- dones: (N_Eps, 12) —— 标记。
- 关键补充: hidden_states: (N_Eps, 1, 2, 128)。我们只需要存储每个Episode起始时刻的隐藏状态(对于固定长度任务, 这通常是全0)。

4.1.2 隐藏状态管理(Hidden State Management)

这是一个极易出错的环节。

- 初始化: 在每个Episode开始时($t = 0$), LSTM的隐藏状态 (h_0, c_0) 必须被重置为全0向量⁸。
- 训练时的展开(Unrolling): 在PPO更新阶段, 我们从Buffer中采样一个Batch的Episode(例如32个)。输入形式为 (32, 12, 67)。我们将这个Batch以及全0的初始隐藏状态输入LSTM。PyTorch的LSTM会自动处理12步的序列展开。
- 为什么不需要截断BPTT(Truncated BPTT): 截断BPTT通常用于无限长的序列(如语言模型或持续控制)。由于本任务 $T = 12$, 非常短, 完全在GPU显存和梯度稳定性的承受范围内。因此, 我们应使用全序列BPTT, 这能保证梯度准确地从第12层的奖励传导回第1层的决策¹¹。

4.2 广义优势估计(GAE)的修正

在序列末端稀疏奖励的情况下, GAE的计算尤为关键。

$$A_t = \delta_t + (\gamma\lambda)A_{t+1}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) * (1 - d_t) - V(s_t)$$

- 掩码处理(Masking): 在 $t = 11$ (即第12层) 时, $d_t = 1$ 。此时 $\delta_{11} = r_{11} - V(s_{11})$ 。这确保了Episode结束后的价值不参与计算。
- 价值传递: 对于 $t < 11$, 奖励 r_t 可能为0。此时优势函数完全依赖于价值函数的差分 $\gamma V(s_{t+1}) - V(s_t)$ 。这意味着Critic网络的准确性直接决定了早期层(如第1-5层) 能否获得正确的策略梯度。如果Critic训练失败, 早期层将处于随机游走状态。

4.3 损失函数与梯度策略

PPO的损失函数保持不变, 但计算时需要注意维度变换。

- 维度展平: 在计算Loss前, 将 (Batch, 12,...) 的数据展平为 (Batch * 12,...), 以便进行向量化的比率计算和裁剪。
- 梯度裁剪(Global Gradient Clipping): 由于LSTM容易出现梯度爆炸, 必须在

optimizer.step() 之前应用 torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5) ⁸。

- 熵正则化：由于动作空间较大 ($3 \times 5 = 15$ 种组合)，建议设置较高的初始熵系数 (如0.05)，并随着训练线性衰减至0.01，以防止过早收敛到次优解。

5. 详细代码实现规划

本节提供核心模块的伪代码逻辑，以指导具体开发。

5.1 LSTM策略网络模型类

Python

```
import torch
import torch.nn as nn
import numpy as np

class LSTMStrategyNetwork(nn.Module):
    def __init__(self, observation_space, action_space_g, action_space_s, hidden_dim=128):
        super().__init__()
        # 1. 嵌入层定义
        self.embed_layer_idx = nn.Embedding(12, 16) # 层索引 0-11
        self.embed_prev_g = nn.Embedding(4, 8) # 0-2是动作, 3是SOS
        self.embed_prev_s = nn.Embedding(6, 8) # 0-4是动作, 5是SOS

        # 2. 全局特征投影
        # 假设原始特征17维 + 预算特征3维 = 20维连续特征
        self.fc_continuous = nn.Sequential(
            nn.Linear(20, 32),
            nn.LayerNorm(32),
            nn.ReLU()
        )

        # 3. LSTM核心
        # 输入维度 = 16 + 8 + 8 + 32 = 64
        self.lstm = nn.LSTM(
            input_size=64,
            hidden_size=hidden_dim,
            num_layers=2,
```



```
        batch_first=True,  
        dropout=0.1  
    )
```

```
    # 4. 策略头与价值头
```

```
    self.actor_head = nn.Sequential(  
        nn.Linear(hidden_dim, 64),  
        nn.Tanh()  
    )  
    self.head_g = nn.Linear(64, 3) # GELU logits  
    self.head_s = nn.Linear(64, 5) # Softmax logits
```

```
    self.critic_head = nn.Sequential(  
        nn.Linear(hidden_dim, 64),  
        nn.Tanh(),  
        nn.Linear(64, 1) # Value  
    )
```

```
    self.apply(self._orthogonal_init)
```

```
def _orthogonal_init(self, layer):  
    if isinstance(layer, (nn.Linear, nn.Conv1d)):  
        nn.init.orthogonal_(layer.weight, gain=np.sqrt(2))  
        if layer.bias is not None:  
            nn.init.constant_(layer.bias, 0.0)  
    # LSTM权重初始化通常需要特殊处理, PyTorch默认初始化在PPO中表现尚可
```

```
def forward(self, x_continuous, layer_indices, prev_actions_g, prev_actions_s, hidden_states=None):  
    """  
    x_continuous: (Batch, Seq, 20)  
    layer_indices: (Batch, Seq)  
    prev_actions_g: (Batch, Seq)  
    prev_actions_s: (Batch, Seq)  
    """  
    batch_size, seq_len, _ = x_continuous.size()  
  
    # 特征嵌入与拼接  
    emb_l = self.embed_layer_idx(layer_indices)  
    emb_pg = self.embed_prev_g(prev_actions_g)  
    emb_ps = self.embed_prev_s(prev_actions_s)  
    feat_c = self.fc_continuous(x_continuous)  
  
    lstm_input = torch.cat([emb_l, emb_pg, emb_ps, feat_c], dim=-1)
```



```
# LSTM前向传播
self.lstm.flatten_parameters()
lstm_out, new_hidden = self.lstm(lstm_input, hidden_states)
```

```
# 解码
actor_feat = self.actor_head(lstm_out)
logits_g = self.head_g(actor_feat)
logits_s = self.head_s(actor_feat)
values = self.critic_head(lstm_out)
```

```
return logits_g, logits_s, values, new_hidden
```

5.2 前向采样逻辑 (Rollout)

在收集数据时，必须维护上一步的动作作为当前的输入。

Python

```
def collect_trajectory(env, model):
    obs = env.reset() # 包含全局特征

    # 初始化状态
    prev_g = torch.tensor([0]) # SOS token
    prev_s = torch.tensor([0]) # SOS token
    hidden = None # 等价于全0初始化

    trajectory =

    for layer_idx in range(12):
        # 构造输入
        current_layer = torch.tensor([[layer_idx]])
        # 获取当前连续特征(包含动态更新的Budget)
        cont_feat = torch.tensor(env.get_features()).unsqueeze(0).unsqueeze(0)

        with torch.no_grad():
            lg, ls, val, hidden = model(cont_feat, current_layer, prev_g, prev_s, hidden)

    # 动作采样
    dist_g = torch.distributions.Categorical(logits=lg)
```

```
dist_s = torch.distributions.Categorical(logits=ls)
action_g = dist_g.sample()
action_s = dist_s.sample()

# 环境步进
next_obs, reward, done, info = env.step(action_g.item(), action_s.item())

# 存储数据...

# 更新上一动作
prev_g = action_g
prev_s = action_s
```

6. 与现有系统的集成与迁移

6.1 约束处理策略的迁移

现有的 PPO_8_GREEDY.py 采用了三阶段课程学习 (Curriculum Learning) 来处理 Loss/Pearson/Spearman 约束。这在LSTM中依然适用，但需要调整奖励函数的形式。

在基准中，约束可能通过硬性的Mask或惩罚来实现。在LSTM中，建议将约束状态显式地编码进 Budget Features 中。

- 特征设计: 输入不仅是“Pearson余量”，而应是“当前层与其理想贡献度的偏差”。
- 奖励塑形 (Reward Shaping):
 - 终端奖励: 在 $t = 12$ 时，如果满足所有约束，奖励
$$R = 1.0 + \alpha(\text{TargetCost} - \text{ActualCost})$$
。如果违反约束，奖励
$$R = -10 \times \text{ViolationMagnitude}$$
。
 - 中间奖励 (Potential-Based): 为了引导LSTM，可以在每一步给出一个小的密集奖励，该奖励基于“Budget Margin的变化率”。例如，如果某一步消耗的Cost过大导致Margin急剧下降，给予负奖励。这有助于Critic学习到状态的价值。

6.2 归一化器的集成

原系统使用了 DisentangledNormalizer。在LSTM架构中，这个归一化器应继续用于处理原始17维特征。对于LSTM输出的价值 (Value)，如前所述，应引入PopArt或独立的RunningMeanStd。

- 注意: 不要对离散的Embedding输入进行归一化。只归一化连续的Context特征。
-

7. 风险评估与故障排除指南

在从MLP迁移到LSTM的过程中，可能会遇到以下特有问题：

7.1 灾难性遗忘与长距离依赖失效

- 症状：模型在第10-12层的决策似乎与第1-3层的决策无关，导致整体策略不连贯。
- 原因：LSTM的遗忘门偏置初始化过低，导致早期信息被过早丢弃。
- 对策：强制将遗忘门偏置初始化为1.0甚至2.0。此外，确保“全局上下文”特征在每一步都输入，不仅仅是在第一步。

7.2 价值网络收敛困难

- 症状：Critic的Loss不下降，或者Explained Variance极低。
- 原因：稀疏奖励导致信用分配(Credit Assignment)困难。
- 对策：
 1. 增加中间奖励：即使最终精度未知，计算成本(Cost)是每一步已知的。可以将Cost的负值作为每一步的惩罚，帮助Critic理解“节约成本”的局部价值。
 2. 调整GAE Lambda：将 λ 调高(如0.98)，使优势计算更多地依赖蒙特卡洛回报(真实收益)而非价值估计，减少偏差。

7.3 过拟合于特定层序

- 症状：模型学会了“第3层总是用4bit”的刻板规则，而不看输入特征。
- 原因：位置嵌入(Positional Embedding)过强。
- 对策：在训练初期对位置嵌入施加Dropout，或者引入一些随机打乱层序的辅助任务(虽然物理上不可行，但作为正则化手段)，迫使模型更多依赖内容特征而非位置特征。

8. 结论

本报告详细规划了将Transformer优化项目的策略网络迁移至LSTM架构的全过程。通过引入序列化建模，我们能够捕捉层级间的量化敏感性依赖，这在理论上优于当前的独立决策模型。

核心变革点总结：

1. 状态重构：从扁平向量转变为包含历史动作嵌入和动态预算状态的序列输入。
2. 全序列BPTT：利用固定长度 $T = 12$ 的特性，进行精确的端到端梯度传播，解决稀疏奖励问题。
3. PopArt价值头：通过自适应归一化解决奖励尺度波动大的问题。

建议开发团队按照“环境接口改造 -> LSTM模型搭建 -> 单元测试(Dummy Task) -> 全量训练”的路径执行迁移。如果在实施过程中遇到性能瓶颈，应优先检查Critic网络的价值预测准确度，因

为它是PPO算法在序列决策中能否成功的基石。

Works cited

1. 10.1. Long Short-Term Memory (LSTM) — Dive into Deep Learning 1.0.3 documentation, accessed February 11, 2026, https://d2l.ai/chapter_recurrent-modern/lstm.html
2. The Complete LSTM Tutorial With Implementation - Analytics Vidhya, accessed February 11, 2026, <https://www.analyticsvidhya.com/blog/2022/01/the-complete-lstm-tutorial-with-implementation/>
3. Understanding LSTM Networks - Colah's Blog, accessed February 11, 2026, <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
4. A Transformer-Based Reinforcement Learning Framework for Sequential Strategy Optimization in Sparse Data - MDPI, accessed February 11, 2026, <https://www.mdpi.com/2076-3417/15/11/6215>
5. Jointly-Learned State-Action Embedding for Efficient Reinforcement Learning - arXiv, accessed February 11, 2026, <https://arxiv.org/pdf/2010.04444>
6. Istm - Uses of Embedding/ Embedding layer in deep learning - Stack Overflow, accessed February 11, 2026, <https://stackoverflow.com/questions/56333914/uses-of-embedding-embedding-layer-in-deep-learning>
7. The 37 Implementation Details of Proximal Policy Optimization - The ICLR Blog Track, accessed February 11, 2026, <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>
8. Preserving Outputs Precisely while Adaptively Rescaling Targets - Google DeepMind, accessed February 11, 2026, <https://deepmind.google/blog/preserving-outputs-precisely-while-adaptively-rescaling-targets/>
9. Learning values across many orders of magnitude, accessed February 11, 2026, <https://arxiv.org/abs/1602.07714>
10. Truncated Backpropagation: A Comprehensive Guide for 2025 - Shadecoder, accessed February 11, 2026, <https://www.shadecoder.com/topics/truncated-backpropagation-a-comprehensive-guide-for-2025>
11. Unbiasing Truncated Backpropagation Through Time - Yann Ollivier, accessed February 11, 2026, <http://www.yann-ollivier.org/rech/pubs/artbp.pdf>